🚙 VEHICLE RENTAL SYSTEM

MERN STACK INTERNSHIP PROJECT REPORT

Submitted By: Trupti Ramteke
Project Duration: 7 Working Days
Project Level: Intermediate
Project Type: Full-Stack Web Application
Project Date: August 2026

🛠️ TECHNOLOGY STACK

MongoDB Atlas • Express.js • React.js • Node.js • Vite • Tailwind CSS • Axios • JWT • bcrypt • Nodemailer • Cloudinary • Swagger/OpenAPI • Vercel

---

1. PROJECT OBJECTIVE

The **Vehicle Rental System** is a full-stack web application developed using the **MERN Stack** as part of a 7-day internship assignment.

The main objective of this project was to develop a practical vehicle rental platform where customers can **browse, search, filter, check availability, calculate rental costs, book vehicles and manage their bookings**.

The Admin panel provides complete control over **vehicles, categories, bookings, customers, vehicle availability and dashboard statistics**.

This project was designed to provide hands-on experience with **frontend development, backend APIs, database management, authentication, authorization, booking logic, validation, debugging and deployment**.

---

2. PROJECT HIGHLIGHTS ⭐

The application provides a complete end-to-end vehicle rental workflow:

👤 Customer Features

✓ Register, Login & Logout
✓ Browse 12+ vehicles
✓ Search vehicles by name, brand or model
✓ Filter by category, location, fuel and transmission
✓ Filter by minimum/maximum price
✓ Sort vehicles
✓ View detailed vehicle information
✓ Select pickup and return dates

✓ View booked dates on a calendar
✓ Check real-time vehicle availability
✓ Automatically calculate rental cost
✓ Create bookings
✓ View booking history
✓ Cancel eligible bookings
✓ Download PDF booking receipt
✓ Reschedule bookings
✓ Submit vehicle reviews and ratings
✓ Receive booking confirmation emails

👨‍💼 Admin Features

✓ Secure Admin Login
✓ Dashboard statistics
✓ Manage vehicles
✓ Add/Edit/Delete vehicles
✓ Change vehicle availability status
✓ Manage vehicle categories
✓ View and manage bookings
✓ Confirm pending bookings
✓ Cancel bookings
✓ Mark bookings as completed
✓ View customer information
✓ Track revenue
✓ Revenue analytics chart

---

3. USER ROLES

👤 CUSTOMER

Customers can:

- Register and login securely
- Browse available vehicles
- Search and filter vehicles
- View vehicle details
- Select pickup and return dates
- Check vehicle availability
- Calculate rental costs
- Create and manage bookings
- View booking history
- Cancel eligible bookings
- Reschedule eligible bookings
- Download booking receipts
- Submit ratings and reviews
- Manage their profile

👨‍💼 ADMIN

Admins can:

- Login securely
- View the Admin Dashboard
- Manage vehicles
- Manage vehicle categories
- Change vehicle status
- View and manage bookings
- Manage customers
- View revenue statistics
- Monitor overall rental activity

Role-Based Authorization ensures that customers cannot access protected Admin pages or APIs.

---

4. AUTHENTICATION & AUTHORIZATION 🔐

The application uses **JWT-based authentication** for secure user access.

Registration Includes:

- Name
- Email
- Phone
- Password
- Confirm Password

Security Features:

✓ Unique email validation
✓ Frontend & backend validation
✓ Password hashing using **bcrypt**
✓ JWT authentication
✓ Role-based authorization
✓ Protected APIs and routes

Passwords are never stored as plain text. The system securely hashes passwords using bcrypt and verifies login credentials using `bcrypt.compare()`.

---

5. LOGIN CREDENTIALS

🔑 ADMIN ACCOUNT

Email: admin@rental.com
Password: Admin@123

👤 CUSTOMER ACCOUNT

Email: user@rental.com
Password: User@123

Note: These credentials are provided only for local/demo testing.

---

6. VEHICLE MANAGEMENT 🚗

The Admin can completely manage the vehicle inventory.

Admin Operations:

✓ Add vehicles
✓ View vehicles
✓ Edit vehicles
✓ Delete vehicles
✓ Change vehicle status
✓ Search vehicles
✓ Filter vehicles

Vehicle Information:

- Vehicle Name
- Brand
- Model
- Category
- Registration Number
- Manufacturing Year
- Fuel Type
- Transmission
- Number of Seats
- Price Per Day
- Location
- Images
- Description
- Features
- Status
- Created Date
- Updated Date

Vehicle Status:

🟢 **AVAILABLE** – Ready for booking
🔴 **UNAVAILABLE** – Temporarily unavailable
🟠 **MAINTENANCE** – Under maintenance

Vehicles marked as **UNAVAILABLE** or **MAINTENANCE** cannot be booked.

7. VEHICLE CATEGORIES

The system supports multiple vehicle categories:

🚙 **Hatchback**
🚗 **Sedan**
🚙 **SUV**
🚐 **MUV**
🏎️ **Luxury**
⚡ **Electric**

Admins can:

✓ Create categories
✓ View categories
✓ Edit categories
✓ Delete categories

---

8. VEHICLE LISTING & SEARCH 🔎

The customer-facing vehicle listing provides a clean and user-friendly browsing experience.

Each vehicle card displays:

- Vehicle image
- Vehicle name
- Brand & model
- Category
- Number of seats
- Fuel type
- Transmission
- Price per day
- Location
- Availability status

Search & Filters

Customers can search/filter vehicles by:

✓ Name
✓ Brand
✓ Model
✓ Category
✓ Location
✓ Fuel Type
✓ Transmission
✓ Minimum Price
✓ Maximum Price

9. VEHICLE DETAILS PAGE

The Vehicle Details page provides complete information before booking.

Displays:

✓ Vehicle image/gallery
✓ Vehicle information
✓ Price per day
✓ Location
✓ Features
✓ Description
✓ Availability
✓ Ratings & Reviews

The page includes a **"Rent This Vehicle"** option to start the booking process.

⭐ Ratings & Reviews

The application also includes a bonus review system where customers can view ratings and submit their own reviews.

Example rating: ⭐ **4.8 / 5**

---

10. RENTAL & AVAILABILITY SYSTEM 🗓️

Customers select:

Pickup Date & Time → Return Date & Time

The system calculates:

Rental Days = Return Date − Pickup Date

Validation Rules:

✓ Pickup date is required
✓ Return date is required
✓ Pickup date cannot be in the past
✓ Return date must be after pickup date
✓ Vehicle must be available
✓ Existing bookings must be checked
✓ Overlapping bookings are rejected

The availability check is performed on the **backend**, preventing customers from bypassing frontend restrictions.

11. BOOKED-DATE CALENDAR 🔴

A major feature of the project is the **booked-date calendar**.

The backend provides booked dates through an API:

GET /api/vehicles/{id}/booked-dates

The frontend uses this information to display already-booked dates as **RED and disabled** on the calendar.

This gives customers a clear visual indication of when a vehicle is unavailable.

---

12. DATE OVERLAP PREVENTION 🚫

The system prevents two customers from booking the same vehicle for overlapping rental periods.

Example:

If an existing booking is:

20 August → 25 August

The following bookings will be rejected:

❌ 19 August → 21 August
❌ 21 August → 23 August
❌ 23 August → 28 August
❌ 20 August → 25 August

Core Logic:

```
newStart < existingEnd && newEnd > existingStart
```

This backend validation ensures that the same vehicle cannot be double-booked.

---

13. BOOKING & RENTAL CALCULATION 💰

After selecting the rental dates, the customer receives a booking summary containing:

- Vehicle
- Pickup Date
- Return Date
- Rental Days
- Price Per Day
- Subtotal
- Total Amount

Example:

Toyota Innova

20 August → 23 August
3 Days × ₹3,500

TOTAL: ₹10,500

The **backend calculates the final rental price** instead of trusting rental days, price or total amount received from the frontend.

This provides better security and prevents manipulation of booking amounts.

---

14. BOOKING MODEL

Each booking contains:

- Booking Number
- User
- Vehicle
- Pickup Date
- Return Date
- Rental Days
- Price Per Day
- Total Amount
- Booking Status
- Payment Status
- Created At
- Updated At

Booking Status:

🟡 **PENDING**
🔵 **CONFIRMED**
🔴 **CANCELLED**
🟢 **COMPLETED**

Payment Status:

PENDING / NOT_APPLICABLE

A real payment gateway is not required for the core project.

---

15. BOOKING CONFIRMATION ✅

After successful booking, customers can view:

✓ Unique Booking Number
✓ Vehicle Details
✓ Pickup & Return Dates
✓ Rental Duration
✓ Total Amount
✓ Booking Status

Example:

✓ BOOKING CONFIRMED

Booking: VR-20260820-0001
Vehicle: Toyota Innova
20 Aug → 23 Aug
3 Days
Total: ₹10,500
Status: Confirmed

Every booking receives a **unique booking number**.

---

16. MY BOOKINGS 📋

Customers can manage their complete booking history.

Booking Dashboard Includes:

5 Total Bookings
2 Active
1 Completed
2 Cancelled

Customers can filter bookings by:

- All
- Upcoming
- Completed
- Cancelled

Example Booking:

Maruti Ertiga
Booking: **VR-20260831-4019**
02/09/2026 → 09/09/2026
7 Days
₹22,400
Wardha
CONFIRMED

Available actions include:

✓ View Details
✓ PDF Receipt
✓ Reschedule
✓ Cancel Booking

---

17. BOOKING CANCELLATION RULE ⏰

A customer can cancel an eligible booking **up to 24 hours before the pickup time**.

If the pickup time is less than 24 hours away:

Cancellation is not allowed.

The rule is enforced on the **backend as well as the frontend**, ensuring that users cannot bypass the restriction.

---

18. ADMIN DASHBOARD 📊

The Admin Dashboard provides a quick overview of the complete rental system.

Dashboard Statistics:

🚘 **Total Vehicles:** 12
🟢 **Available Vehicles:** 12
📋 **Active Bookings:** 3
👥 **Total Customers:** 4
💰 **Total Revenue:** ₹38,500

Cancelled bookings are excluded from the revenue calculation.

Recent Bookings Table

The dashboard displays:

Booking | Customer | Vehicle | Amount | Status

A **Revenue Analytics Chart** is also implemented as a bonus feature.

---

19. ADMIN VEHICLE MANAGEMENT

The Admin vehicle management page provides:

12 Total Vehicles
12 Available
0 Rented

Admins can:

✓ Search vehicles
✓ Filter by status
✓ Edit vehicle details
✓ Delete vehicles
✓ Change availability status

Example vehicle:

BMW X5 xDrive40i
Brand: BMW
Year: 2024
Price/Day: ₹12,000
Status: AVAILABLE

---

20. ADMIN BOOKING MANAGEMENT

The Admin can monitor and manage all bookings.

Booking Statistics:

11 Total Bookings
₹38,500 Revenue
0 Pending
3 Confirmed
1 Completed
7 Cancelled

Admin actions:

✓ View bookings
✓ Search bookings
✓ Filter bookings
✓ Confirm bookings
✓ Cancel bookings
✓ Mark bookings as completed

MongoDB `populate()` is used to display actual customer and vehicle information instead of only database IDs.

21. CUSTOMER MANAGEMENT 👥

The Admin can view registered customer information.

Customer details include:

- Name
- Email
- Phone
- Number of Bookings
- Registration Date
- Account Status

Example:

Trupti Ramteke
user@rental.com
5 rides
₹47,500 revenue contribution

---

22. USER INTERFACE & RESPONSIVENESS 📱

The application is designed to work across:

💻 Desktop
📱 Mobile
📲 Tablet

The UI includes:

✓ Loading states
✓ Error states
✓ Empty states
✓ Form validation
✓ Success notifications
✓ Error notifications
✓ Confirmation dialogs
✓ Responsive layouts

The overall design focuses on being **clean, simple, consistent and user-friendly**.

---

23. CLOUDINARY IMAGE MANAGEMENT ☁️

Vehicle images are managed using **Cloudinary**.

Cloudinary provides reliable image hosting and allows vehicle images to be stored and displayed using secure URLs.

This improved the vehicle image management compared to storing unreliable image paths.

---

24. EMAIL CONFIRMATION 📧

The application integrates **Nodemailer** for booking confirmation emails.

After successful booking, the system can send booking-related confirmation information to the customer's registered email address.

---

25. PDF BOOKING RECEIPT 📄

A **PDF booking receipt** has been implemented as a bonus feature.

Customers can access the receipt from the **My Bookings** section.

The receipt contains important booking information such as:

- Booking Number
- Customer
- Vehicle
- Pickup Date
- Return Date
- Rental Duration
- Total Amount
- Booking Status

---

26. SWAGGER API DOCUMENTATION 🧪

The backend API is documented using **Swagger/OpenAPI**.

API Documentation:

http://localhost:5000/api-docs

Swagger provides an interactive interface where APIs can be viewed and tested using **"Try it out"** functionality.

This made API development and testing easier during the project.

---

27. BONUS FEATURES IMPLEMENTED ⭐

In addition to the mandatory requirements, several bonus features were implemented:

✓ Cloudinary Image Upload
✓ Nodemailer Email Confirmation
✓ PDF Booking Receipt
✓ Revenue Analytics Chart
✓ Vehicle Ratings & Reviews
✓ Google Maps Integration
✓ Coupon/Discount – WELCOME10
✓ Booking Reschedule
✓ Swagger/OpenAPI Documentation

These additional features enhanced the functionality and made the application more similar to a real-world vehicle rental platform.

---

28. IMPORTANT PROBLEMS & SOLUTIONS 🛠️

Problem 1: Vehicle Images Were Not Displaying

Initially, vehicle image links were not working correctly.

Solution: I corrected the image paths and implemented proper image management using **Cloudinary secure URLs**.

Problem 2: Dashboard Statistics Were Showing Incorrect Values

Some dashboard statistics such as customer count and revenue were initially showing incorrect or zero values.

Solution: I implemented proper backend queries using MongoDB operations such as `countDocuments()` and aggregation to calculate:

- Total users
- Total vehicles
- Available vehicles
- Active bookings
- Revenue

The calculated data was then provided to the frontend through APIs.

Problem 3: Customer and Vehicle Names Were Not Displaying in Bookings

Initially, only MongoDB IDs were displayed instead of customer and vehicle names.

Solution: I used MongoDB's: `populate('user')` and `populate('vehicle')` to retrieve related information and display meaningful customer and vehicle details.

Problem 4: Login Password Validation Issue

There was an issue with password validation during login.

Solution: I used: `bcrypt.compare()` to securely compare the entered password with the hashed password stored in MongoDB. I also handled email addresses consistently using: `toLowerCase().trim()`

Problem 5: Double Booking Issue

A vehicle could potentially be requested for overlapping dates.

Solution: I implemented backend overlap validation along with the **Booked Dates API** and disabled booked dates on the frontend calendar.This ensures that the same vehicle cannot be booked by multiple customers for overlapping periods.

Problem 6: Cancellation Rule Enforcement

Customers needed to be prevented from cancelling bookings within 24 hours of pickup.

Solution: I implemented the 24-hour cancellation rule on both frontend and backend.

If: **Pickup Time − Current Time < 24 hours** the booking cannot be cancelled.

Problem 7: Vercel Deployment API Error

The project worked correctly on localhost but initially produced API errors after deployment.

Solution:

I checked and corrected:

✓ Frontend API URL
✓ Environment variables
✓ CORS configuration
✓ MongoDB Atlas network access
✓ Vercel deployment configuration

I also learned to use the **browser Network tab and deployment logs** to identify and resolve API issues.

---

29. BACKEND ARCHITECTURE 🔄

The application follows a structured backend request flow:

Request → Route → Middleware → Controller → Model → Database → Response

This architecture helped maintain separation between different responsibilities and made the backend easier to understand and debug.

---

30. PROJECT DATA / SEED DATA

Sample data was provided for testing.

Includes:

✓ 1 Admin Account
✓ 1 Customer Account
✓ 12+ Vehicles
✓ Multiple Vehicle Categories
✓ Sample Bookings
✓ Customer Data
✓ Vehicle Reviews

This makes it easier to test both Customer and Admin workflows.

---

31. HOW TO RUN THE PROJECT 💻

BACKEND / SERVER

Open the terminal:

```
cd server
```

Then start the backend:

```
node server.js
```

Backend:

http://localhost:5000

Swagger Documentation:

http://localhost:5000/api-docs

FRONTEND / CLIENT

Open another terminal:

```
cd client
```

Then start the frontend:

```
npm start
```

Frontend:

http://localhost:3000

The frontend communicates with the backend using the configured API URL.

---

32. PROJECT VERIFICATION

During development, the backend was successfully tested with the following configuration:

MongoDB: Connected ✅
Cloudinary: Configured ✅
Environment Variables: Loaded ✅
Server: Running on Port 5000 ✅
Swagger Documentation: Available ✅

The server successfully established a connection with MongoDB Atlas and started the application backend.

---

33. WHAT I LEARNED 🎓

Through this project, I gained practical experience in:

✓ MERN Stack Development
✓ React.js
✓ Node.js & Express.js
✓ MongoDB & MongoDB Atlas
✓ REST API Development
✓ JWT Authentication
✓ Role-Based Authorization
✓ Password Security using bcrypt
✓ Backend Validation
✓ Vehicle Availability Logic
✓ Date Overlap Prevention
✓ Secure Backend Price Calculation
✓ MongoDB `populate()`
✓ Nodemailer

✓ Cloudinary
✓ Swagger API Documentation
✓ Admin Dashboard Development
✓ Debugging
✓ Vercel Deployment
✓ Environment Variables
✓ CORS Configuration

---

34. MY PROJECT EXPERIENCE 💡

This project gave me practical experience in developing a **complete full-stack web application from frontend to backend and database**.

Before this project, I mainly worked on smaller frontend projects. While developing the Vehicle Rental System, I worked with both frontend and backend technologies and gained a better understanding of how different parts of a real-world application communicate with each other.

The most challenging areas were **authentication, MongoDB operations, REST APIs, booking availability, date-overlap prevention, backend price calculation and deployment**.

By identifying and solving these problems, I improved my:

✓ Problem-solving skills
✓ Debugging ability
✓ Backend understanding
✓ API development skills
✓ Database knowledge
✓ Full-stack development confidence

This project helped me understand not only **how to build features**, but also **how to identify, debug and solve real development problems**.

---

35. CONCLUSION 🚀

The **Vehicle Rental System** is a complete full-stack web application developed using the **MERN Stack** as a 7-day internship assignment.

The application provides customers with an easy way to **browse, search, filter, check availability, calculate rental costs, make bookings and manage their rental history**.

The Admin panel provides complete control over **vehicles, categories, customers, bookings, vehicle status and revenue statistics**.

The project demonstrates important real-world development concepts including:

JWT Authentication • Role-Based Authorization • bcrypt Password Security • REST APIs • MongoDB • Backend Validation • Vehicle Availability • Date Overlap Prevention • Secure Rental Calculation • Booking Management • Responsive UI • Cloudinary • Nodemailer • Swagger • Deployment

Overall, this project significantly improved my understanding of **full-stack web development, backend architecture, database operations, API integration, debugging and deployment**.

📌 PROJECT DETAILS

Detail	Information
Project Name	Vehicle Rental System
Project Type	MERN Stack Internship Assignment
Duration	7 Working Days
Level	Intermediate
Frontend	React.js, Vite, Tailwind CSS, Axios
Backend	Node.js, Express.js
Database	MongoDB Atlas
Authentication	JWT
Password Security	bcrypt
Email Service	Nodemailer
Image Management	Cloudinary
API Documentation	Swagger/OpenAPI
Deployment	Vercel
Project Date	August 2026

👩‍💻 MADE BY

TRUPTI RAMTEKE

MERN STACK INTERNSHIP PROJECT

August 2026

🚙 VEHICLE RENTAL SYSTEM — FROM IDEA TO FULL-STACK IMPLEMENTATION 🚙